

RSNA Knee Abnormality Detection

Project guide for your first Kaggle competition and Sprint 03 | written October 2, 2026

Goal of this guide. Give you one clear project direction, the reasoning behind each decision, and a day-by-day path to a working submission before the October 22 deadline (11:59 PM UTC). Winning is not the goal. A complete, honest, well-documented pipeline is, and that is also a strong portfolio piece.

1. The project in one sentence

Use the radiology reports to create training labels for thousands of unlabeled knee MRI studies, then train an image-only model on those labels and check it honestly on the 58 studies that have real labels.

Why this is the right shape: the test set has no reports (about 1,300 studies, images only), but your own checks show that only 58 of 4,407 training studies have labels. The reports are therefore the only way to get a large training set. Text is used at training time, images are used at prediction time.

2. What your checks proved

Finding from your notebook	Number	What it means for the project
Training studies / series	4,407 / 24,371	About 5.5 series per study (3 to 14): pick the useful ones, do not read all.
Studies with real labels	58 (all 12 labels)	Too few to train on. Keep them as your gold evaluation set.
Studies with report only	4,349	Your report-labeler must create labels for these.
Report languages (detected)	en 1,736; es 682; tr 546; hr 406; el 321; de 262; bg 220; nl 153; fr 81	Top 5 languages cover about 84% of reports. Rules must be multilingual.
Series planes	Sagittal 9,864; Coronal 8,609; Axial 5,898	Use all three planes per study.
Fluid_Sensitive vs Fat_Suppression	Identical in train	One flag is enough. Prefer fluid-sensitive series.
DICOM format in 6,000 sampled series	100% Explicit VR Little Endian	Easy decoding. Still keep a try/except fallback.
Slices per series	median 30, min 11, max 320	Sample a fixed number (16) of central slices.
Image sizes	256 up to 1024 pixels	Resize everything to one size (192 or 224).
Hardware	2x Tesla T4, 4 CPU cores, 33.7 GB RAM, 19.5 GB output	Cache compact uint8 arrays, plan for about 8 GB.

Note: language names come from an automatic detector (langdetect). The competition text mentions 12 languages, so a few labels (for example Croatian vs similar languages) may be imperfect. Treat the counts as approximate.

3. Pipeline overview

Stage	Input	Output	Where it runs
A. Report labeler	4,407 reports	12 pseudo-labels per study	Training time only
B. Labeler check	Pseudo-labels vs the 58 real labels	Per-finding precision / recall	Training time only
C. Preprocessing	DICOM series	16 slices x up to 3 planes, uint8 arrays	Training time (cached)
D. Image model	Slices per study	12 probabilities	Training and final inference
E. Final evaluation	58 gold studies	Macro AUC with confidence interval	Training time only
F. Inference notebook	test_series + weights	submission.csv	Kaggle, Internet OFF, up to 9 h

4. Stage A: the report labeler (the heart of the project)

Your first Spanish report already shows the idea: "*Rotura de menisco interno ... Artrosis femorotibial medial. Derrame.*" reads as Medial Meniscus = 1, Medial OA = 1, Effusion = 1. A labeler turns that kind of sentence into the 12 columns.

Three levels, in order. Stop whenever the agreement with the 58 gold labels is good enough.

- **Level 1: keyword rules with negation (start here).** One list of patterns per finding and language, plus a check that the sentence does not say the opposite ("no fracture", "ACL intact"). Fast, explainable, and you will learn the data.
- **Level 2: multilingual embeddings + classifier.** Sentence embeddings of the report and logistic regression. With only 58 labels, use it as a second opinion or train it on Level 1 labels, not as the main labeler.
- **Level 3 (optional): a small instruction-tuned LLM** run on the T4 to read each report and answer 12 yes/no questions. Check the Models tab for what can be attached offline. Compare against gold before trusting it.

Starter cue table (verify against real reports, then extend to tr, hr, el, bg, nl, fr)

Finding	English	Spanish	German	Turkish
ACL	anterior cruciate ligament	ligamento cruzado anterior	vorderes Kreuzband	ön çapraz bağ
Medial meniscus	medial meniscus	menisco interno	Innenmeniskus	iç menisküs
Lateral meniscus	lateral meniscus	menisco externo	Außenmeniskus	dış menisküs
Effusion	joint effusion	derrame articular	Gelenkerguss	eklem efüzyonu
Baker's cyst	Baker's cyst	quiste de Baker	Baker-Zyste	Baker kisti
Bone contusion	bone contusion / bruise	contusión ósea	Knochenkontusion	kemik kontüzyonu
Fracture	fracture	fractura	Fraktur	kırık

```
import re
NEG = r"\b(no|not|without|absent|intact|normal|sin|keine|kein|yok)\b"
def has_finding(report, patterns):
    for clause in re.split(r"[.;\n]| but | pero | aber ", report.lower()):
        if any(re.search(p, clause) for p in patterns):
            if not re.search(NEG, clause):
                return 1
    return 0
```

This sketch is only a starting point. Always use word boundaries (\b), otherwise "no" matches inside "interno". Expect problems with negation scope, "tear" vs "degeneration", and previous surgery. Reading 20 to 30 reports per language and keeping a list of mistakes is the best use of your time.

5. Stage B: how to trust the labeler

Compare the labeler's output with the 58 real labels, per finding. Report precision, recall and F1, and look at every disagreement by hand.

- With 58 studies, rare findings have few positives (for example MCL is about 15%, roughly 9 studies), so numbers will be noisy. Say so in your report instead of hiding it.
- The gold set is also your only honest test. **Do not train on the 58 studies** until the very end, and do not tune everything to them. Use a bootstrap to show a confidence interval around the final AUC.
- Prevalence in the 58 (Effusion 60%, Synovitis 47%, Medial Meniscus 45%, ACL 41%) may differ from the full data, and the competition itself warns that prevalence can differ between train, public and private test sets.

6. Stage C: preprocessing

Goal: turn each study into a small, fast-to-load array. Reading every slice would take 2 to 5 hours on one thread, so reduce first.

Decision	Choice	Reason
Series per study	Up to 3: one per plane, fluid-sensitive preferred	Covers all anatomy, keeps the data small.
Slices per series	16, evenly spaced across the central 60% of the stack	Pathology is usually near the center of the knee.
Slice order	Sort by InstanceNumber (or ImagePositionPatient) from the header	File names are UUIDs, so directory order is NOT slice order. Check which tags exist (only 86 are kept).
Intensity	Apply rescale slope if present, clip at 1st and 99th percentile per series, scale to 0-255	Scanners differ a lot.
Size	Resize to 192x192	About 8 GB total as uint8: 4,407 x 3 x 16 x 192 x 192.
Speed	4 worker processes, run as a committed notebook	About 211k slice reads: roughly 10 to 25 minutes with 4 cores (9 to 25 ms per read).
Persistence	Save the output as a Kaggle Dataset	Sessions are temporary. A Dataset survives and can be re-attached.

```
def pick_series(study_df):
    # study_df = rows of train_series for ONE study
    picks = []
    for plane in ["Sagittal", "Coronal", "Axial"]:
        cand = study_df[study_df.Anatomical_Plane == plane]
        fluid = cand[cand.Fluid_Sensitive == 1]
        use = fluid if len(fluid) else cand
        if len(use):
            picks.append(use.iloc[0].SeriesInstanceUID)
    return picks

def central_idx(n, k=16):
    lo, hi = int(0.2 * n), int(0.8 * n)
    return np.linspace(lo, max(lo, hi - 1), k).round().astype(int)
```

7. Stage D: the image model

Keep the first model simple. A simple model that runs end to end beats a clever one that never reaches submission.

Part	Baseline	Possible upgrade (only after baseline works)
Backbone	Pretrained 2D CNN from timm (resnet18 or efficientnet_b0), ImageNet weights	Larger backbone, or one backbone per plane
Input	Each slice repeated to 3 channels, shared backbone for all slices	Stack 3 neighbouring slices as the 3 channels
Combine slices	Mean pooling over slices, then concatenate the 3 planes	Attention pooling over slices
Head	Linear layer with 12 outputs and sigmoid	Separate heads per finding group
Loss	BCEWithLogits, pos_weight for rare findings	Soft labels or label smoothing for noisy pseudo-labels
Training	AdamW, mixed precision (torch.autocast), 5 to 10 epochs, one T4	Augmentation (flips, small shifts), 5-fold on pseudo-labels
Metric	Macro AUC on the 58 gold studies + per-finding AUC	Bootstrap confidence interval

Pretrained weights must be available without Internet in the final notebook. Download them once while Internet is on, save the state dict, and attach it as a Kaggle Dataset or Model. The competition rules (as shown in your screenshot) allow freely and publicly available external data, including pretrained models.

Honest expectations

The image model learns from machine-made labels, so it inherits their mistakes. An AUC in a modest range is a normal result for a first attempt. What makes the project strong is the story you can tell: how good your labeler is per finding, which findings the images predict well, and which fail and why.

8. Stage F: the offline submission notebook

This is where first-time competitors lose days, so build it early and test it often.

- Settings: Internet **OFF**, GPU on, output file named exactly `submission.csv` with StudyInstanceUID plus the 12 columns in the sample order.
- Inputs to attach: the competition data, your weights Dataset, and a wheels Dataset. Your notebook already created `/kaggle/working/wheels` with `pylibjpeg` packages. Save that folder as a Dataset and install with `pip install --no-index --find-links`.
- Read `test.csv` and `test_series.csv` from the competition folder. The files you see now are only 3 examples. The real test set (about 1,300 studies) replaces them during scoring.
- Wrap each study in `try/except`. If anything fails, output the average training probabilities for that study so the run never crashes.

- Time check: 1,300 studies x 3 series x 16 slices is about 62k slice reads, far below the 9-hour limit.

Smoke test first. Within the next day, submit a tiny notebook that writes 0.5 for every label (like `sample_submission.csv`). It proves the whole submit pipeline works long before you have a model.

9. Day-by-day plan (about 3 hours per day)

Assumed: you can give around 3 hours a day. If you have less, drop the optional items in the "Improve" rows, never the smoke test or the offline notebook.

Dates	Task	Done when
Oct 2-3	Accept rules (deadline Oct 15, do it now). Smoke-test submission with 0.5 values, Internet OFF.	A score appears on the leaderboard
Oct 3-5	NB1: read 20-30 reports per language, language table, keyword lists for the top 5 languages.	Cue sheet for 12 findings
Oct 5-7	NB2: rule labeler v1, compare with the 58 gold studies per finding.	Precision/recall table
Oct 6-8	NB3 (in parallel): preprocessing job running in the background, saved as a Dataset.	About 8 GB cached
Oct 8-10	NB4: baseline model trained on pseudo-labels, first macro AUC on gold.	First number logged
Oct 10-13	Improve the labeler (negation, more languages, optional embeddings or LLM check). Retrain.	Gold agreement improves
Oct 13-16	At most 3 model experiments (backbone, attention pooling, image size). Keep a results table.	Results table
Oct 16-19	Final training, bootstrap confidence intervals, save weights as a Dataset.	Final weights + metrics
Oct 19-20	NB5: offline inference notebook, full test run, submit.	Real score on the leaderboard
Oct 21	Buffer: fix problems, optional second submission, write README and report.	Report draft
Oct 22	Deadline 11:59 PM UTC. No new experiments.	Nothing pending

10. How to stay active and organized

- **Daily routine (about 3 h):** 20 minutes reading the Discussion and Code tabs, 2 hours building, 15 minutes writing a log entry.
- **Keep a `PROJECT_LOG.md`:** date, what you tried, result, next step. It becomes the backbone of your Sprint 03 report.
- **Commit often:** use Save Version in Kaggle after every working step, and push notebooks to GitHub (one repo, one folder per notebook).
- **Name notebooks clearly:** 01_reports_eda, 02_report_labeler, 03_preprocess, 04_train, 05_inference.
- **Watch your GPU quota** in Kaggle settings, and turn the accelerator off when you are only reading data or writing code.
- **Read the Rules tab** for team size and daily submission limits. Do not share private code outside your team. Public discussion is the normal place to share ideas.

11. Common traps (check this list before each submission)

Trap	How to avoid it
Slices out of order	Sort by InstanceNumber or position from the header, never by file name.
Training on the gold 58	Keep them out of training and tuning until the final step.
Report-only information at test time	The image model must not need any text. Reports are used only for labels.
Negation mistakes in the labeler	Test on sentences like "no fracture" and "ACL intact" in every language.
Different image sizes and intensities	Resize to one size and normalize per series.
Missing decoder at submission time	Attach the wheels Dataset and install with --no-index. Test with Internet OFF.
Wrong output file or columns	Name it submission.csv, keep column order, include the header, no extra index column.
Notebook crashes on one bad study	Per-study try/except with fallback probabilities.
Overfitting to a tiny validation set	Report confidence intervals, avoid more than a few experiments scored on the 58.
Leaving everything to the last day	Offline notebook working by Oct 19, final upload by Oct 21.

12. What to hand in for Sprint 03

Nolyth's exact requirements are not known to me, so adapt this list. A strong submission would include:

- A GitHub repository with the five notebooks, a clear README, and a short architecture diagram.
- A results table: labeler agreement per finding on the 58 gold studies, and the image model's macro AUC with a confidence interval.
- Error analysis: three examples where the labeler failed and three where the image model failed, with your explanation.
- A learning section that names what you practiced: weak supervision, multi-label classification, macro AUC, transfer learning, multiple-instance pooling over slices, validation with a tiny gold set, and offline code-competition submission.
- Your final Kaggle notebook link and leaderboard score.

13. Your next 24 hours

#	Action
1	Accept the competition rules on the Rules tab (if not done).
2	Create a notebook that reads test.csv and writes submission.csv with 0.5 everywhere. Internet OFF, then Submit.
3	Save the /kaggle/working/wheels folder as a Kaggle Dataset (needed for offline decoding later).
4	Create the GitHub repo and PROJECT_LOG.md. Paste your two check outputs into the log.
5	Start NB1: print 5 reports each for en, es, tr, hr, el next to the gold labels (where available) and note the patterns.

Ask me anytime. Good next requests: "write NB1", "help me build the keyword lists for Turkish", "write the preprocessing notebook", or "review my labeler results". Paste outputs or errors and I will work from them.